

Задание 1.1. Создание проекта в Cursor

Общая характеристика редактора Cursor

Cursor это редактор кода, созданный на основе Visual Studio Code и дополненный встроенным AI-агентом, который интегрирован не как отдельное расширение, а как полноценная часть рабочего процесса. Главная идея Cursor заключается в том, что искусственный интеллект получает прямой доступ к структуре и содержимому проекта и может не просто подсказывать код, а самостоятельно анализировать задачи, вносить изменения сразу в нескольких файлах и выполнять команды в терминале.

В отличие от классических «помощников по автодополнению», Cursor ориентирован на работу с проектом в целом: редактор индексирует файлы открытой папки, строит представление о зависимостях между ними и использует эту информацию при формировании ответов. Благодаря этому AI-агент способен учитывать контекст всего приложения, а не только открытого в данный момент файла.

Cursor позиционируется как инструмент для ускорения повседневной разработки: написания нового кода, рефакторинга, поиска и исправления ошибок, а также изучения незнакомых участков кода. Он подходит как для индивидуальной разработки, так и для командной работы, поскольку сохраняет привычный интерфейс и структуру проектов VS Code.

Системные требования и установка

Для корректной работы Cursor заявлены следующие минимальные требования к системе:

- операционная система Windows, macOS или Linux;
- оперативная память не менее 4 ГБ (для комфортной работы с крупными проектами рекомендуется 8 ГБ и более);
- свободное место на диске не менее 2 ГБ;
- стабильное подключение к интернету, так как обработка запросов к AI-модели выполняется на удалённых серверах, а не на локальном компьютере.

Отдельного видеоускорителя (GPU) для работы редактора не требуется все вычисления, связанные с языковой моделью, выполняются на стороне сервиса, а локальный компьютер отвечает только за интерфейс редактора и индексацию файлов проекта.

Установка выполняется стандартным образом: с официального сайта [cursor.com](https://cursor.sh) загружается установочный файл под используемую операционную систему, после чего запускается обычный мастер установки. После первого запуска пользователю предлагается авторизация через email, Google или GitHub без входа в аккаунт AI-функции редактора недоступны. При переходе с VS Code настройки и установленные плагины можно перенести автоматически с помощью опции «Import from VS Code».

Создание проекта

Новый проект в Cursor создаётся так же, как и в VS Code: через меню File → Open Folder, где выбирается папка с кодом. После открытия папки Cursor индексирует структуру проекта анализирует файлы и связи между ними и делает эту информацию доступной AI-агенту для дальнейшей работы в Agent Mode, Chat и режиме Cmd+K.

Кодинг

Основной режим работы Agent Mode: достаточно описать задачу текстом, и AI самостоятельно анализирует код, создаёт файлы, пишет код и запускает команды. Для точечных правок используется инлайн-режим Cmd+K (macOS) или Ctrl+K (Windows) выделяется фрагмент, вводится инструкция, AI применяет изменение на месте. Ссылки @ дают точный контекст: @file.ts конкретный файл, @functionName функция, @docs документация. При написании кода AI предсказывает следующие строки, принимаемые по Tab.

Форматирование

Cursor наследует настройки форматирования VS Code. В settings.json включается editor.formatOnSave: true для автоформата при сохранении. Стил настраивается в Settings | Editor | Code Style. При наличии файла .editorconfig в проекте IDE автоматически применяет указанные в нём правила.

Отладка

Для диагностики ошибок используется чат с контекстом терминала: копируется stack trace, добавляется ссылка @terminal, задаётся вопрос «почему падает и как исправить» Cursor сопоставляет ошибку с кодом проекта и предлагает конкретное решение. Инлайн-команды // debug: why is this throwing NullPointerException? и // lint: fix ESLint warnings позволяют исправлять ошибки и стилевые замечания без переключения контекста. Стандартные точки останова и панель переменных работают как в VS Code.

Запуск и компиляция

Cursor автоматически выполняет терминальные команды, но по умолчанию запрашивает подтверждение. Для C/C++ создаётся tasks.json с командой компиляции (g++ -std=c++17 -g \${file} -o \${fileDirname}/\${fileBasenameNoExtension}) и launch.json для отладки через GDB. Компиляция запускается по Ctrl+Shift+B, запуск через панель Run или кнопку в терминале. Для других языков используются штатные конфигурации VS Code.

Git: коммит и push

Cursor полностью наследует возможности интеграции с Git от VS Code. В левой панели активности находится значок системы контроля версий: там отображаются изменённые файлы, их можно добавить в индекс, ввести сообщение коммита и нажать «Commit». После

коммита интерфейс предложит синхронизироваться с удалённым репозиторием подтверждение завершает Push.

В Cursor есть и полезная функция: AI-генерация сообщения коммита. В поле ввода сообщения через палитру команд или пользовательское сочетание клавиш можно вызвать `cursor.generateGitCommitMessage` AI составит корректное сообщение коммита на основе внесённых изменений и выполнять весь процесс одним нажатием.

При первой настройке удалённого репозитория рекомендуется использовать SSH-ключ, а не HTTPS-авторизацию. SSH настраивается один раз: после добавления публичного ключа push больше не требует повторного ввода учётных данных, что избавляет от повторяющихся сбоев браузерной авторизации.

AI-чат (Chat)

AI-чат находится на боковой панели и по умолчанию вызывается через `Cmd+I` (macOS) или `Ctrl+I` (Windows/Linux). Его назначение понимать код, отлаживать проблемы, обсуждать архитектурные решения, а не напрямую править код.

Chat автоматически подхватывает в качестве контекста текущий открытый файл, позицию курсора, выделенный код и недавно просмотренные файлы. Он также поддерживает ссылки через символ `@` на конкретные файлы, папки, документацию или веб-страницы. Когда нужно сначала разобраться, «как связана эта логика аутентификации», а уже потом решать, что менять, Chat самый подходящий вход. Он позволяет выбирать разные базовые модели (например, семейство Claude, GPT), а собственный Composer от Cursor отличается низкой задержкой и подходит для быстрых итераций.

Cmd+K встроенное редактирование

`Cmd+K` (в Windows/Linux `Ctrl+K`) самый характерный для Cursor способ «изменить здесь и сейчас». Выделив фрагмент кода и нажав это сочетание, вы получаете поле ввода: опишите изменение естественным языком, и AI сразу сгенерирует diff-превью на месте, а после подтверждения правка применится без переключения в окно чата.

Область применения чёткая: небольшие, точно очерченные правки. Например: «перепиши эту функцию в стиле `async/await`», «добавь обработку ошибок в этот цикл», «переименуй переменную и обнови все ссылки в текущей области видимости». Если же нужно искать и править сразу в нескольких файлах и прогонять тесты это уже задача Agent-режима (боковая панель `Cmd+I`) или Chat, а не `Cmd+K`.

Разделение Chat и `Cmd+K` можно сформулировать так: Chat решает задачу «разобраться», `Cmd+K` задачу «поправить». В связке они покрывают весь путь от понимания до внесения изменений, не превращая каждую мелкую правку в полнорепозиторный поиск по контексту. Операции Git (коммит, push) при этом выполняются через встроенный интерфейс системы контроля версий, унаследованный от VS Code, без переключения в терминал.

Сравнение Cursor и Visual Studio Code

Поскольку Cursor построен на кодовой базе VS Code, оба редактора внешне и по логике работы очень похожи: одинаковая структура интерфейса, боковая панель проводника, встроенный терминал, система расширений и большинство привычных сочетаний клавиш. Именно поэтому переход с VS Code на Cursor не требует переучивания предусмотрена даже отдельная опция «Import from VS Code», которая переносит настройки, темы и установленные расширения в новый редактор.

Основное отличие проявляется на уровне встроенного искусственного интеллекта. В VS Code какие-либо AI-функции по умолчанию отсутствуют и появляются только после установки сторонних расширений, например GitHub Copilot; при этом такие расширения, как правило, ограничиваются подсказками по автодополнению кода в пределах одного файла. В Cursor AI-агент встроен изначально и работает на уровне всего проекта: в режиме Agent Mode он способен самостоятельно создавать файлы, вносить правки сразу в нескольких местах и выполнять терминальные команды по текстовому описанию задачи, что выходит далеко за рамки простого автодополнения.

Ещё одно отличие способ обращения к контексту при работе с AI. В Cursor реализована система ссылок с символом «@», позволяющая явно указать модели, какой файл, функцию или документацию следует учитывать при ответе, а также быстрый доступ к чату с контекстом терминала для разбора ошибок и точечный режим правок Cmd+K. В обычном VS Code подобных встроенных механизмов нет: их приходится реализовывать через настройки конкретного стороннего расширения, и глубина интеграции с проектом обычно ниже.

При этом важно отметить, что Cursor сохраняет полную совместимость с большинством расширений из магазина VS Code, поэтому пользователь не теряет привычную функциональность редактора, а лишь дополняет её. С точки зрения выбора инструмента можно сформулировать простое правило: если в работе важна глубокая AI-поддержка на уровне всего проекта целесообразнее использовать Cursor; если требуется лёгкий и полностью настраиваемый редактор без встроенного AI либо с собственным набором расширений оправдан выбор классического VS Code.